

COVID-19 ANALYTICS

Data description

A centralised repository of up-to-date and curated datasets on or related to the spread and characteristics of the novel coronavirus (SARS-CoV-2) and its associated illness, COVID-19. Globally, there are several efforts underway to gather this data, and we are working with partners to make this crucial data freely available and keep it up-to-date. Hosted on the AWS cloud, we have seeded our curated data lake with COVID-19 case tracking data from Johns Hopkins and The New York Times, hospital bed availability from Definitive Healthcare, and over 45,000 research articles about COVID-19 and related coronaviruses from the Allen Institute for AI.

Data is available here

<https://registry.opendata.aws/aws-covid19-lake/>

Scope of project

Create tables that contains:

- **Dimensions**
 - `dim_date` — calendar(date id(fulldate) , year, month, day)
 - `dim_state` — US states (state code and statefull name)
- **Facts (daily, state-level)**
 - `fact_cases_state_daily` — [cumulative cases/deaths & new cases/deaths]
 - `fact_testing_state_daily` – [cumulative tests,positivity & new tests, positivity]

Datasets (Bronze state)

S3 layout :

s3://<My_BUCKET>/covid/

bronze/ {nytimes|covid_tracking|static}/...

silver/ {cases_standardized | testing_standardized} /...

gold/ {dim_date | dim_state | fact_cases_state_daily | fact_testing_state_daily} /...

The screenshots show the Amazon S3 console interface for the 'covid-19-analytics-project2' bucket. The first screenshot shows the 'bronze/' folder with three subfolders: 'covid_tracking/', 'nytimes/', and 'static/'. The second screenshot shows the 'gold/' folder with four subfolders: 'dim_date/', 'dim_state/', 'fact_cases_state_daily/', and 'fact_testing_state_daily/'. The third screenshot shows the 'silver/' folder with two subfolders: 'cases_standardized/' and 'testing_standardized/'.

Name	Type
covid_tracking/	Folder
nytimes/	Folder
static/	Folder

Name	Type
dim_date/	Folder
dim_state/	Folder
fact_cases_state_daily/	Folder
fact_testing_state_daily/	Folder

Name	Type
cases_standardized/	Folder
testing_standardized/	Folder

S3 Bucket contains raw csv

- nytimes/us_states.csv (or JHU state rollups) → cases/deaths
- covid_tracking/states_daily.csv → tests positive/negative/total
- static/states_abv.csv → map state name ↔ abbreviation

1) Bronze → Silver (standardize to Parquet)

1.1 What implemented

- Read CSVs from / **bronze** /
- Normalize and standardized:
 - **full_date** as DATE
 - **state_code** (2-char) and **state_name** (proper case) via **states_abv.csv**
 - Cast numerics to **BIGINT/INT**
- Write partitioned **Parquet** to **silver** / with partitions: **state_code**, **year**, **month**, **day**
- Register Silver in **Glue** and validate in **Athena**

Glue spark for bronze etl job:

☰

AWS Glue

Getting started

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Zero-ETL integrations [New](#)

Data Catalog

Catalogs

Databases

Tables

Stream schema registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

Data Integration and ETL

Zero-ETL integrations [New](#)

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Interactive Sessions

Data classification tools

Sensitive data detection

Record Matching

Triggers

Workflows (orchestration)

Blueprints

Security configurations

Cost management [New](#)

Legacy pages

What's New [↗](#)

Documentation [↗](#)

AWS Marketplace

Enable compact mode

Enable new navigation

covidetljob

Script

Job details

Runs

Data quality

Schedules

Version Control

Script

Info

```
1 import sys
2 from awsglue.utils import getResolvedOptions
3 from awsglue.context import GlueContext
4 from awsglue.job import Job
5 from pyspark.context import SparkContext
6
7 from pyspark.sql import functions as F
8
9 # -----
10 # Job Initialization
11 # -----
12
13 args = getResolvedOptions(sys.argv, ['JOB_NAME'])
14
15 sc = SparkContext()
16 glueContext = GlueContext(sc)
17 spark = glueContext.spark_session
18
19 job = Job(glueContext)
20 job.init(args['JOB_NAME'], args)
21
22 # -----+-----
23 # Paths
24 # -----
25
26 BUCKET = "covid-19-analytics-project2"
27
28 BRONZE = f"s3://{BUCKET}/covid/bronze"
29 SILVER = f"s3://{BUCKET}/covid/silver"
30
31 # -----
32 # Helper Functions
33 # -----
34
35 upper_trim = lambda c: F.upper(F.trim(F.col(c)))
36
37 # -----
38 # Lookup Table
39 # -----
40
41 states = (
42     spark.read
43     .option("header", True)
44     .csv(f"{BRONZE}/static/states_abv.csv")
45     .select(
46         upper_trim("Abbreviation").alias("state_code"),
47         F.initcap("State").alias("state_name")
48     )
49 )
50
51 # =====
52 # CASES STANDARDIZED
53 # =====
54
55 cases_raw = (
56     spark.read
57     .option("header", True)
58     .csv(f"{BRONZE}/nytimes/us_states.csv")
59 )
60
61 cases_std = (
62     cases_raw
63     .withColumn("full_date", F.to_date("date"))
64     .withColumn("state_name_raw", F.initcap(F.col("state")))
65     .join(
```

Python Ln 22, Col 36 0 Errors: 0 0 Warnings: 0

AWS Glue

Getting started

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Zero-ETL integrations [New](#)

Data Catalog

Catalogs

Databases

Tables

Stream schema registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

Data Integration and ETL

Zero-ETL integrations [New](#)

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Interactive Sessions

Data classification tools

Sensitive data detection

Record Matching

Triggers

Workflows (orchestration)

Blueprints

Security configurations

Cost management [New](#)

Legacy pages

What's New

Documentation

AWS Marketplace

☒ Enable compact mode

☒ Enable new navigation

covidetljob

ScriptJob detailsRunsData qualitySchedulesVersion Co

ScriptInfo

```
65 .join(
66 states,
67 states.state_name == F.col("state_name_raw"),
68 "left"
69 )
70 .withColumn("cases_cum", F.col("cases").cast("long"))
71 .withColumn("deaths_cum", F.col("deaths").cast("long"))
72 .withColumn("year", F.year("full_date"))
73 .withColumn("month", F.month("full_date"))
74 .withColumn("day", F.dayofmonth("full_date"))
75 .select(
76 "full_date",
77 "state_code",
78 "state_name",
79 "cases_cum",
80 "deaths_cum",
81 "year",
82 "month",
83 "day"
84 )
85 .dropna(subset=["full_date", "state_code"])
86 )
87 #
88 # write case_data into silver in partitioned
89 (
90 cases_std.write
91 .mode("overwrite")
92 .partitionBy(
93 "state_code",
94 "year",
95 "month",
96 "day"
97 )
98 .parquet(f"{SILVER}/cases_standardized")
99 )
100
101 # =====
102 # TESTING data STANDARDIZED
103 # =====
104
105 tests_raw = (
106 spark.read
107 .option("header", True)
108 .csv(f"{BRONZE}/covid_tracking/states_daily.csv")
109 )
110
111 tests_std = (
112 tests_raw
113 .withColumn(
114 "full_date",
115 F.to_date(
116 F.col("date").cast("string"),
117 "yyyyMMdd"
118 )
119 )
120 .withColumn(
121 "state_code",
122 upper_trim("state")
123 )
124 .join(
125 states.select(
126 "state_code",
127 "state_name"
128 ),
129 "state_code",
```

PythonLn 22, Col 36Errors: 0Warnings: 0

AWS Glue

Getting started

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Zero-ETL integrations [New](#)

▼ Data Catalog

Catalogs

Databases

Tables

Stream schema registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

▼ Data Integration and ETL

Zero-ETL integrations [New](#)

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Interactive Sessions

Data classification tools

Sensitive data detection

Record Matching

Triggers

Workflows (orchestration)

Blueprints

Security configurations

Cost management [New](#)

► Legacy pages

What's New

Documentation

AWS Marketplace

Enable compact mode

Enable new navigation

covidetljob

Script

Job details

Runs

Data quality

Schedules

Script Info

```

118 )
119 )
120 .withColumn(
121   "state_code",
122   upper_trim("state")
123 )
124 .join(
125   states.select(
126     "state_code",
127     "state_name"
128   ),
129   "state_code",
130   "left"
131 )
132 .withColumn(
133   "tests_total_cum",
134   F.col("totalTestResults").cast("long")
135 )
136 .withColumn(
137   "tests_pos_cum",
138   F.col("positive").cast("long")
139 )
140 .withColumn(
141   "tests_neg_cum",
142   F.col("negative").cast("long")
143 )
144 .withColumn("year", F.year("full_date"))
145 .withColumn("month", F.month("full_date"))
146 .withColumn("day", F.dayofmonth("full_date"))
147 .select(
148   "full_date",
149   "state_code",
150   "state_name",
151   "tests_total_cum",
152   "tests_pos_cum",
153   "tests_neg_cum",
154   "year",
155   "month",
156   "day"
157 )
158 .dropna(
159   subset=[
160     "full_date",
161     "state_code"
162   ]
163 )
164 )
165 #
166 #WRITE TESTING DATA INTO SIVER IN PARTITIONED FORM
167 (
168   tests_std.write
169     .mode("overwrite")
170     .partitionBy(
171       "state_code",
172       "year",
173       "month",
174       "day"
175     )
176     .parquet(f"{SILVER}/testing_standardized")
177 )
178
179 print("Silver complete.")
180
181 job.commit()
182

```

Python

Ln 22, Col 36

⊙ Errors: 0

⚠ Warnings: 0

Glue/Athena register and validate Silver

```

CREATE EXTERNAL TABLE IF NOT EXISTS
covid_silver_db.cases_standardized (
  full_date date,
  state_code string,

```

```

    state_name string,
    cases_cum bigint,
    deaths_cum bigint,
    year int,
    month int,
    day int
)
PARTITIONED BY (state_code string, year int, month int, day int)
STORED AS PARQUET
LOCATION 's3://YOUR_BUCKET/covid/silver/cases_standardized/';
MSCK REPAIR TABLE covid_silver_db.cases_standardized;

```

-- Testing

```

CREATE EXTERNAL TABLE IF NOT EXISTS
covid_silver_db.testing_standardized (
    full_date date,
    state_code string,
    state_name string,
    tests_total_cum bigint,
    tests_pos_cum bigint,
    tests_neg_cum bigint,
    year int, month int, day int
)
PARTITIONED BY (state_code string, year int, month int, day int)
STORED AS PARQUET
LOCATION 's3://YOUR_BUCKET/covid/silver/testing_standardized/';
MSCK REPAIR TABLE covid_silver_db.testing_standardized;

```

,

-- ATHENA CHECK

Query 1 : (+) ▼

```
1 SELECT MIN(full_date), MAX(full_date), COUNT(*) FROM covid_silver_db.cases_standardized;
```

SQL Ln 1, Col 89

Run again ▼ Explain ↗ Cancel Clear Create ▼

Reuse query results up to 60 minutes ago

Query results | Query stats

Completed Time in queue: 70 ms Run time: 38.609 sec Data scanned: 1.42 MB

Results (1) Copy Download results CSV

Search rows

#	_col0	_col1	_col2
1	2020-01-21	2022-06-14	42606

Query 1 : (+)

```
1 SELECT state_code, COUNT(*) FROM covid_silver_db.testing_standardized GROUP BY 1 ORDER BY 2 DESC;
```

SQL Ln 1, Col 98

Run again ▼ Explain ↗ Cancel Clear Create ▼

Reuse query results up to 60 minutes ago

Query results | Query stats

Completed Time in queue: 24.065 sec Run time: 22.518 sec Data scanned: 1.42 MB

Results (0) Copy Download results CSV

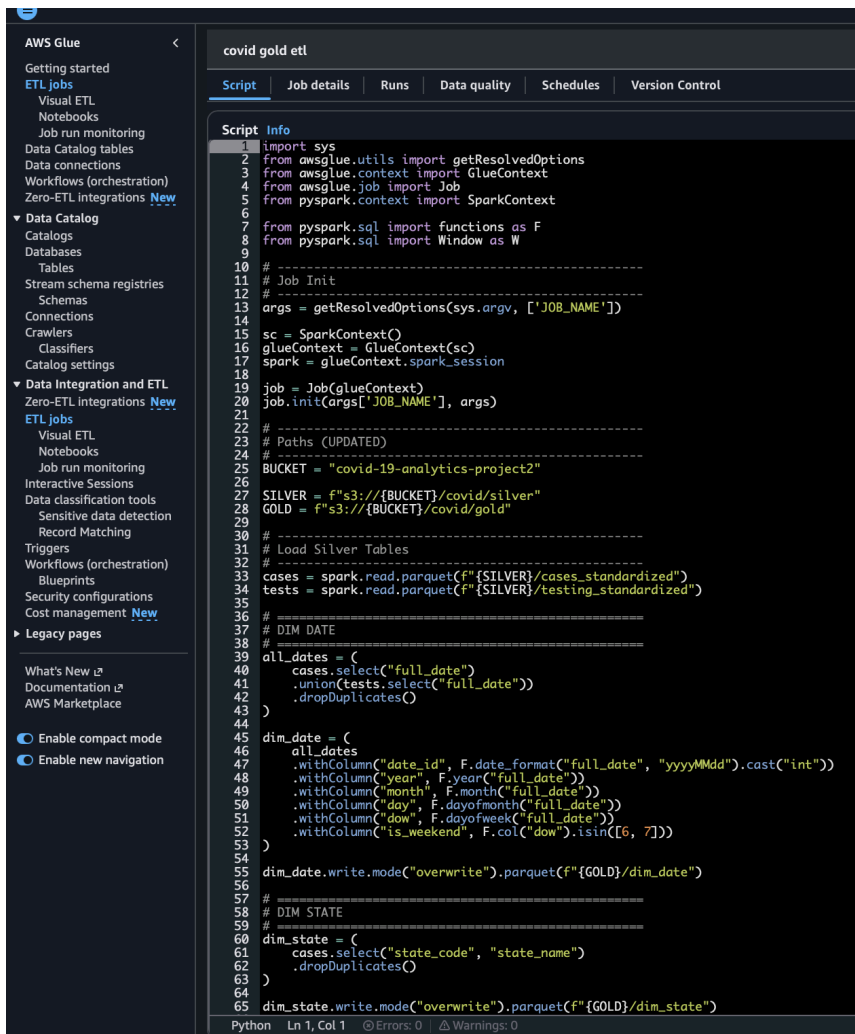
Search rows

#	state_code	_col1
1	WA	420
2	MA	411
3	VA	406
4	FL	404
5	NJ	392

2) Silver → Gold (star schema)

2.1 Model (simple)

- **dim_date**: date_id (YYYYMMDD), full_date, year, month, day, dow, is_weekend
- **dim_state**: state_code (PK), state_name
- **fact_cases_state_daily**: date_id, state_code, cases_cum, deaths_cum, new_cases, new_deaths
- **fact_testing_state_daily**: date_id, state_code, tests_total_cum, tests_pos_cum, tests_neg_cum, new_tests, positivity_rate



```
covid gold etl

Script | Job details | Runs | Data quality | Schedules | Version Control

Script Info
1 import sys
2 from awsglue.utils import getResolvedOptions
3 from awsglue.context import GlueContext
4 from awsglue.job import Job
5 from pyspark.context import SparkContext
6
7 from pyspark.sql import functions as F
8 from pyspark.sql import Window as W
9
10 # -----
11 # Job Init
12 # -----
13 args = getResolvedOptions(sys.argv, ['JOB_NAME'])
14
15 sc = SparkContext()
16 glueContext = GlueContext(sc)
17 spark = glueContext.spark_session
18
19 job = Job(glueContext)
20 job.init(args['JOB_NAME'], args)
21
22 # -----
23 # Paths (UPDATED)
24 # -----
25 BUCKET = "covid-19-analytics-project2"
26
27 SILVER = f"s3://{BUCKET}/covid/silver"
28 GOLD = f"s3://{BUCKET}/covid/gold"
29
30 # -----
31 # Load Silver Tables
32 # -----
33 cases = spark.read.parquet(f"{SILVER}/cases_standardized")
34 tests = spark.read.parquet(f"{SILVER}/testing_standardized")
35
36 # -----
37 # DIM DATE
38 # -----
39 all_dates = (
40     cases.select("full_date")
41     .union(tests.select("full_date"))
42     .dropDuplicates()
43 )
44
45 dim_date = (
46     all_dates
47     .withColumn("date_id", F.date_format("full_date", "yyyyMMdd").cast("int"))
48     .withColumn("year", F.year("full_date"))
49     .withColumn("month", F.month("full_date"))
50     .withColumn("day", F.dayofmonth("full_date"))
51     .withColumn("dow", F.dayofweek("full_date"))
52     .withColumn("is_weekend", F.col("dow").isin([6, 7]))
53 )
54
55 dim_date.write.mode("overwrite").parquet(f"{GOLD}/dim_date")
56
57 # -----
58 # DIM STATE
59 # -----
60 dim_state = (
61     cases.select("state_code", "state_name")
62     .dropDuplicates()
63 )
64
65 dim_state.write.mode("overwrite").parquet(f"{GOLD}/dim_state")

Python Ln 1, Col 1 | Errors: 0 | Warnings: 0
```

Getting started

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Zero-ETL integrations

New

Data Catalog

Catalogs

Databases

Tables

Stream schema registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

Data Integration and ETL

Zero-ETL integrations

New

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Interactive Sessions

Data classification tools

Sensitive data detection

Record Matching

Triggers

Workflows (orchestration)

Blueprints

Security configurations

Cost management

New

Legacy pages

What's New

↗

Documentation

↗

AWS Marketplace

Enable compact mode

Enable new navigation

covid gold etl

Script

Job details

Runs

Data quality

Schedules

Version Control

Script

Info

```
62 )
63 )
64 )
65 dim_state.write.mode("overwrite").parquet(F"{GOLD}/dim_state")
66
67 # =====
68 # FACT CASES
69 # =====
70 w = W.partitionBy("state_code").orderBy("full_date")
71
72 fact_cases = (
73     cases
74     .withColumn("date_id", F.date_format("full_date", "yyyyMMdd").cast("int"))
75     .withColumn(
76         "new_cases",
77         F.greatest(
78             F.col("cases_cum") - F.lag("cases_cum").over(w),
79             F.lit(0)
80         )
81     )
82     .withColumn(
83         "new_deaths",
84         F.greatest(
85             F.col("deaths_cum") - F.lag("deaths_cum").over(w),
86             F.lit(0)
87         )
88     )
89     .select(
90         "date_id",
91         "state_code",
92         "cases_cum",
93         "deaths_cum",
94         "new_cases",
95         "new_deaths"
96     )
97 )
98
99 fact_cases.write.mode("overwrite") \
100     .partitionBy("state_code") \
101     .parquet(F"{GOLD}/fact_cases_state_daily")
102
103 # =====
104 # FACT TESTING
105 # =====
106 w2 = W.partitionBy("state_code").orderBy("full_date")
107
108 fact_tests = (
109     tests
110     .withColumn("date_id", F.date_format("full_date", "yyyyMMdd").cast("int"))
111     .withColumn(
112         "new_tests",
113         F.greatest(
114             F.col("tests_total_cum") - F.lag("tests_total_cum").over(w2),
115             F.lit(0)
116         )
117     )
118     .withColumn(
119         "positivity_rate",
120         F.when(
121             F.col("tests_total_cum") > 0,
122             (F.col("tests_pos_cum") / F.col("tests_total_cum")).cast("double")
123         )
124     )
125     .select(
126         "date_id",
127         "state_code"
128     )
129 )
130
131 fact_tests.write.mode("overwrite") \
132     .partitionBy("state_code") \
133     .parquet(F"{GOLD}/fact_testing_state_daily")
134
135 # =====
136 # Gold layer completed successfully.
137 # =====
138 job.commit()
```

Python Ln 1, Col 1

Errors: 0

Warnings: 0

Interactive Sessions

Data classification tools

Sensitive data detection

Record Matching

Triggers

Workflows (orchestration)

Blueprints

Security configurations

Cost management

New

Legacy pages

What's New

↗

Documentation

↗

AWS Marketplace

Enable compact mode

Enable new navigation

103 # =====

104 # FACT TESTING

105 # =====

106 w2 = W.partitionBy("state_code").orderBy("full_date")

107

108 fact_tests = (

109 tests

110 .withColumn("date_id", F.date_format("full_date", "yyyyMMdd").cast("int"))

111 .withColumn(

112 "new_tests",

113 F.greatest(

114 F.col("tests_total_cum") - F.lag("tests_total_cum").over(w2),

115 F.lit(0)

116)

117)

118 .withColumn(

119 "positivity_rate",

120 F.when(

121 F.col("tests_total_cum") > 0,

122 (F.col("tests_pos_cum") / F.col("tests_total_cum")).cast("double")

123)

124)

125 .select(

126 "date_id",

127 "state_code",

128 "tests_total_cum",

129 "tests_pos_cum",

130 "new_tests",

131 "positivity_rate"

132)

133)

134)

135

136 fact_tests.write.mode("overwrite") \

137 .partitionBy("state_code") \

138 .parquet(F"{GOLD}/fact_testing_state_daily")

139

140 # =====

141 # Gold layer completed successfully.

142 # =====

143 job.commit()

Python Ln 1, Col 1

Errors: 0

Warnings: 0

2.3 Glue/Athena — register Gold

```
CREATE DATABASE IF NOT EXISTS covid_gold_db;
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS covid_gold_db.dim_date (  
    date_id int,  
    full_date date,  
    year int,  
    month int,  
    day int,  
    dow int,  
    is_weekend boolean  
)  
STORED AS PARQUET  
LOCATION 's3://covid-19-analytics-project2/covid/gold/dim_date/';
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS covid_gold_db.dim_state (  
    state_code string,  
    state_name string  
)  
STORED AS PARQUET  
LOCATION 's3://covid-19-analytics-project2/covid/gold/dim_state/';
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS covid_gold_db.fact_cases_state_daily (  
    date_id int,  
    state_code string,  
    cases_cum bigint,  
    deaths_cum bigint,  
    new_cases int,  
    new_deaths int  
)  
PARTITIONED BY (state_code string)  
STORED AS PARQUET  
LOCATION 's3://covid-19-analytics-project2/covid/gold/fact_cases_state_daily/';  
MSCK REPAIR TABLE covid_gold_db.fact_cases_state_daily;
```

```
CREATE EXTERNAL TABLE IF NOT EXISTS covid_gold_db.fact_testing_state_daily (  
    date_id int,  
    state_code string,  
    tests_total_cum bigint,  
    tests_pos_cum bigint,  
    tests_neg_cum bigint,  
    new_tests int,  
    positivity_rate double  
)
```

PARTITIONED BY (state_code string)
STORED AS PARQUET
LOCATION 's3://covid-19-analytics-project2/covid/gold/fact_testing_state_daily/';
MSCK REPAIR TABLE covid_gold_db.fact_testing_state_daily;

Check athena gold database:

The screenshot shows the Amazon Athena Query Editor interface. The SQL query is `select* from covid_gold_db.dim_date limit 5;`. The query has been executed successfully, as indicated by the green 'Completed' status. The results are displayed in a table with 8 columns: #, date_id, full_date, year, month, day, dow, and is_weekend. The results show 5 rows of data.

#	date_id	full_date	year	month	day	dow	is_weekend
1	20210929	2021-09-29	2021	9	29	4	false
2	20200619	2020-06-19	2020	6	19	6	true
3	20201227	2020-12-27	2020	12	27	1	false
4	20211202	2021-12-02	2021	12	2	5	false

The screenshot shows the Amazon Athena Query Editor interface. The SQL query is `select* from covid_gold_db.dim_state limit 3;`. The query has been executed successfully, as indicated by the green 'Completed' status. The results are displayed in a table with 3 columns: #, state_code, and state_name. The results show 3 rows of data.

#	state_code	state_name
1	DC	District Of Columbia
2	NC	North Carolina
3	SC	South Carolina

Amazon Athena > Query editor

Recent queries | Save queries | Query settings | Workgroup: primary

Query 1

```
1 select* from covid_gold_db.fact_cases_state_daily limit 3;
```

SQL Ln 1, Col 59

Run again Explain Cancel Clear Create

Reuse query results up to 60 minutes ago

Query results | Query stats

Completed Time in queue: 87 ms Run time: 704 ms Data scanned: 41.90 KB

Results (3) Copy Download results CSV

Search rows

#	date_id	cases_cum	deaths_cum	new_cases	new_deaths	state_code
1	20200301	2	0	0	0	RI
2	20200302	2	0	0	0	RI
3	20200303	2	0	0	0	RI

Amazon Athena > Query editor

Recent queries | Save queries | Query settings | Workgroup: primary

Query 1

```
1 select* from covid_gold_db.fact_testing_state_daily limit 3;
```

SQL Ln 1, Col 52

Run again Explain Cancel Clear Create

Reuse query results up to 60 minutes ago

Query results | Query stats

Completed Time in queue: 100 ms Run time: 694 ms Data scanned: 3.00 KB

Results (3) Copy Download results CSV

Search rows

#	date_id	tests_total_cum	tests_pos_cum	tests_neg_cum	new_tests	positivity_rate	state
1	20200305	6	0	0	0	0.0	DC
2	20200306	8	0	0	2	0.0	DC
3	20200307	8	0	0	0	0.0	DC

REDSHIFT

New namespace and workgroup

General information		
Workgroup covid19analytics	Date created June 12, 2026, 17:46 (UTC-05:00)	Endpoint covid19analytics.039914330453.us-east-2.redshift-serverless.amazonaws.com:5439/dev
Namespace covidnamespace	Price-performance target 50 (Balanced)	JDBC URL jdbc:redshift://covid19analytics.039914330453.us-east-2.redshift-serverless.amazonaws.com:5439/dev Download driver
Workgroup ARN arn:aws:redshift-serverless:us-east-2:039914330453:workgroup/8bc77cf2-efcf-41a6-a980-22f5550d9c13	Custom domain name -	ODBC URL Driver={Amazon Redshift (x64)}; Server=covid19analytics.039914330453.us-east-2.redshift-serverless.amazonaws.com; Database=dev
Workgroup version 1.0.323213 ↗	Patch version Patch 202 ↗	
Autonomics configuration - new -	Track - new Current	
Status Available		

Query data

Redshift query editor v2

Create

Load data

Filter resources

Serverless: covid19analytics

native databases (2)

dev

covid_gold

Tables 4

di...

di...

fa...

f...

Views 0

Funci... 0

Stored...0

public

sample_data_dev

external databases (1)

Run

Limit 100

Explain

Isolated session

Serverless: covid19analytics

```
1 CREATE SCHEMA IF NOT EXISTS covid_gold;
2
3 CREATE TABLE IF NOT EXISTS covid_gold.dim_date (
4     date_id int PRIMARY KEY,
5     full_date date,
6     year smallint,
7     month smallint,
8     day smallint,
9     dow smallint,
10    is_weekend boolean
11 ) SORTKEY(date_id);
12
13 COPY covid_gold.dim_date
14 FROM 's3://covid-19-analytics-project2/covid/gold/dim_date/'
15 IAM_ROLE 'arn:aws:iam::039914330453:role/redshift-role'
16 FORMAT AS PARQUET;
17
18
19 CREATE TABLE IF NOT EXISTS covid_gold.dim_state (
20     state_code varchar(2) PRIMARY KEY,
21     state_name varchar(64)
22 ) DISTSTYLE ALL;
23
24 COPY covid_gold.dim_state
25 FROM 's3://covid-19-analytics-project2/covid/gold/dim_state/'
26 IAM_ROLE 'arn:aws:iam::039914330453:role/redshift-role'
27 FORMAT AS PARQUET;
28
29 SELECT * from covid_gold.dim_state;
```

Redshift query editor v2

Filter resources

Serverless: covide19analytics

native databases (2)

dev

covid_gold

Tables 4

di...

di...

fa...

f...

Views 0

Funci... 0

Stored...0

public

sample_data_dev

external databases (1)

Run Limit 100 Explain Isolated session Serverless: co... dev

```
36 deaths_cum bigint,
37 new_cases int,
38 new_deaths int
39 ) DISTKEY(state_code) SORTKEY(date_id, state_code);
40
41 COPY covid_gold.fact_cases_state_daily
42 FROM 's3://covid-19-analytics-project2/covid/gold/fact_cases_state_daily/'
43 IAM_ROLE 'arn:aws:iam::039914330453:role/redshift-role'
44 FORMAT AS PARQUET;
45
46
47 CREATE TABLE IF NOT EXISTS covid_gold.fact_testing_state_daily (
48   date_id int NOT NULL REFERENCES covid_gold.dim_date(date_id),
49   state_code varchar(2) NOT NULL REFERENCES covid_gold.dim_state(state_code),
50   tests_total_cum bigint,
51   tests_pos_cum bigint,
52   tests_neg_cum bigint,
53   new_tests int,
54   positivity_rate double precision
55 ) DISTKEY(state_code) SORTKEY(date_id, state_code);
56
57 COPY covid_gold.fact_testing_state_daily
58 FROM 's3://covid-19-analytics-project2/covid/gold/fact_testing_state_daily/'
59 IAM_ROLE 'arn:aws:iam::039914330453:role/redshift-role'
60 FORMAT AS PARQUET;
61
62
```